

Declaración de variables en Bash

En Bash no es obligatorio declarar una variable antes de usarla. Basta con asignarle un valor:

```
nombre="Yassin"
echo "$nombre"
```

Sin embargo, `declare` permite definir atributos además de asignar un valor:

```
declare [opciones] [nombre[=valor] ...]
```

También puede escribirse como `typeset` en Bash.

Opciones principales de `declare`

Opción	Descripción
<code>-a</code>	Declara un array indexado mediante enteros.
<code>-A</code>	Declara un array asociativo indexado mediante cadenas.
<code>-i</code>	Declara la variable como entera. Las operaciones aritméticas se evalúan automáticamente.
<code>-l</code>	Convierte el contenido de la variable a minúsculas.
<code>-u</code>	Convierte el contenido de la variable a mayúsculas.
<code>-r</code>	Declara la variable como de solo lectura. No se puede modificar ni borrar.
<code>-x</code>	Exporta la variable para que esté disponible en los procesos hijos.
<code>-p</code>	Muestra la declaración y los atributos de la variable.
<code>-f</code>	Muestra o trabaja con funciones del shell.
<code>-F</code>	Muestra los nombres de las funciones y, cuando es posible, su ubicación.

Ejemplos de atributos

Variables enteras

```
declare -i contador
contador=10 + 5
echo "$contador" # 15
```

Conversión a minúsculas y mayúsculas

```
declare -l mes="ENERO"
echo "$mes"           # enero

mes="FEBRERO"
echo "$mes"           # febrero

declare -u nombre="yassin"
echo "$nombre"        # YASSIN
```

El atributo se conserva cuando se asignan nuevos valores.

Variables de solo lectura

```
declare -r dia="jueves"
echo "$dia"

# Esta asignación produce un error:
# dia="viernes"
```

Mostrar la declaración

```
declare -i numero=25
declare -p numero
```

La salida incluye el atributo y el valor, por ejemplo:

```
declare -i numero="25"
```

Arrays

Array indexado

Un array indexado utiliza posiciones enteras, empezando normalmente en cero:

```
declare -a colores
colores[0]="rojo"
colores[1]="verde"
colores[2]="azul"

echo "${colores[0]}"
echo "${colores[@]}"
```

```
echo "${!colores[@]}" # índices
echo "${#colores[@]}" # número de elementos
```

También puede declararse directamente:

```
colores=(rojo verde azul)
```

Array asociativo

Un array asociativo utiliza cadenas como índices:

```
declare -A usuario
usuario[nombre]="Yassin"
usuario[rol]="estudiante"

echo "${usuario[nombre]}"
echo "${!usuario[@]}" # claves
```

También puede declararse un valor de vector asociativo entre paréntesis mediante pares clave valor (primero clave y luego valor):

```
usuario=(nombre Yassin rol estudiante)
```

Hay un par de operadores útiles para trabajar con arrays:

Operador	Significado
<code>\${array[@]}</code>	Todos los elementos del array.
<code>\${array[*]}</code>	Todos los elementos del array, separados por espacios.
<code>\${!array[@]}</code>	Todos los índices del array.
<code>\${#array[@]}</code>	El número de elementos del array.

Funciones

Exportar variables

Una variable normal solo existe en el shell actual. Con `declare -x` se exporta y pasa a estar disponible para los procesos hijos, como otros scripts u órdenes:

```
declare -x mensaje="hola"
./otro-script.sh
```

También puede exportarse después:

```
mensaje="hola"  
export mensaje
```

Un script hijo puede leerla así:

```
echo "$mensaje"
```

Exportar una variable no modifica automáticamente el entorno de la terminal padre. La variable se hereda desde el shell que ejecuta el proceso hacia sus procesos hijos.

Parámetros especiales del shell

Estos parámetros son proporcionados automáticamente por Bash cuando se ejecuta un script:

Parámetro	Significado
<code>\$0</code>	Nombre o ruta del shell script que se está ejecutando.
<code>\$@</code>	Lista de todos los argumentos. Con " <code>\$@</code> ", cada argumento se conserva como un elemento independiente.
<code>\$*</code>	Lista de todos los argumentos. Con " <code>\$*</code> ", se unen usando el primer carácter de <code>\$IFS</code> .
<code>\$#</code>	Número total de argumentos, del 1 al <code>N</code> .
<code>\$1, \$2, ...</code>	Valores de los argumentos posicionales, del primero al <code>N</code> .
<code>\$_</code>	Último argumento de la última orden ejecutada.
<code>\$?</code>	Valor de retorno de la última orden ejecutada. Normalmente, <code>0</code> indica éxito.
<code>\$\$</code>	PID del proceso que ejecuta el script.

Ejemplo

Si ejecutamos:

```
./argumentos2.sh -v 10 "informe final.txt"
```

los parámetros principales contienen:

```
$0 = ./argumentos2.sh  
$1 = -v  
$2 = 10
```

```
$3 = informe final.txt  
$# = 3
```

Diferencia entre "\$@" y "\$*"

"\$@" conserva los argumentos separados. Es la forma recomendada para recorrer o reenviar argumentos:

```
for argumento in "$@"; do  
    echo "$argumento"  
done
```

"\$*" une todos los argumentos en una sola cadena. El separador es el primer carácter de **\$IFS**.

Por defecto, **\$IFS** significa **Internal Field Separator** y normalmente contiene espacio, tabulador y salto de línea. Se utiliza para dividir palabras cuando Bash realiza expansión y separación de campos.

\$? y **\$_** cambian después de ejecutar cada orden. Si se necesita conservar su valor, hay que guardarlo inmediatamente en otra variable.